

Retail Sales Analysis (Project 2 - DAT 301) Anand Chamoli October 9, 2025

This project analyzes the Sample Superstore dataset to uncover insights into sales performance, profitability, and customer behavior across product categories and regions. Using Python, I applied data cleaning, transformation, exploratory analysis, and visualization techniques to answer key business questions.

What are we researching?

1. Which product categories and subcategories drive the most sales and profit?
2. How do sales vary by region and state?
3. Are there seasonal or monthly sales fluctuations?
4. Which customer segments generate the most revenue?
5. How do discounts affect profit margins?

```
In [52]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.ticker import FuncFormatter

%matplotlib inline

pd.set_option('display.max_columns', 20)
plt.style.use('seaborn-v0_8')
```

```
In [61]: df = pd.read_csv("/Users/anandchamoli/Desktop/DATA SCIENCE 2026/DAT 301/Supe
print("Shape:", df.shape)
df.head(5)
```

Shape: (9994, 21)

Out [61]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Co
0	1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	U
1	2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	U
2	3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	U
3	4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	U
4	5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	U

5 rows x 21 columns

Data Cleaning Summary:

1. Converted Order Date and Ship Date columns to datetime format for time-based analysis.
2. Verified no missing values in key columns (Sales, Category, Profit).
3. Removed duplicates and formatted column names for readability.
4. Checked data types and ensured numeric fields were consistent.

```
In [62]: df.info()  
df.isna().sum()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Row ID                 9994 non-null   int64
1   Order ID               9994 non-null   object
2   Order Date             9994 non-null   object
3   Ship Date              9994 non-null   object
4   Ship Mode              9994 non-null   object
5   Customer ID            9994 non-null   object
6   Customer Name          9994 non-null   object
7   Segment                9994 non-null   object
8   Country                9994 non-null   object
9   City                   9994 non-null   object
10  State                  9994 non-null   object
11  Postal Code            9994 non-null   int64
12  Region                 9994 non-null   object
13  Product ID             9994 non-null   object
14  Category               9994 non-null   object
15  Sub-Category           9994 non-null   object
16  Product Name           9994 non-null   object
17  Sales                  9994 non-null   float64
18  Quantity               9994 non-null   int64
19  Discount               9994 non-null   float64
20  Profit                 9994 non-null   float64
dtypes: float64(3), int64(3), object(15)
memory usage: 1.6+ MB

```

```

Out[62]: Row ID                0
         Order ID              0
         Order Date            0
         Ship Date             0
         Ship Mode             0
         Customer ID           0
         Customer Name         0
         Segment              0
         Country              0
         City                 0
         State                0
         Postal Code          0
         Region              0
         Product ID           0
         Category             0
         Sub-Category         0
         Product Name         0
         Sales                0
         Quantity            0
         Discount            0
         Profit              0
         dtype: int64

```

```

In [70]: df = df.drop_duplicates() #Removing duplicates

# Converting Date columns to Datetime

df['Order Date'] = pd.to_datetime(df['Order Date'])

```

```

df['Ship Date'] = pd.to_datetime(df['Ship Date'])

# Add year, month, and week columns

df['Year'] = df['Order Date'].dt.year
df['Month'] = df['Order Date'].dt.month_name()
df['Month_Num'] = df['Order Date'].dt.month

monthly_sales = df.groupby(['Year', 'Month'])['Sales'].sum().reset_index()
monthly_sales

# Checking for negative or invalid values

print("Any negative sales?", (df['Sales'] < 0).any())
df.head(1)

```

Any negative sales? False

Out[70]:

	Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	
0	1	CA- 2016- 152156	2016- 11-08	2016- 11-11	Second Class	CG-12520	Claire Gute	Consumer	United States	Her

1 rows x 24 columns

In [63]: df.describe(include='all').T

Out [63]:

	count	unique	top	freq	mean	std	min
Row ID	9994.0	NaN	NaN	NaN	4997.5	2885.163629	1.0
Order ID	9994	5009	CA-2017-100111	14	NaN	NaN	NaN
Order Date	9994	1237	9/5/2016	38	NaN	NaN	NaN
Ship Date	9994	1334	12/16/2015	35	NaN	NaN	NaN
Ship Mode	9994	4	Standard Class	5968	NaN	NaN	NaN
Customer ID	9994	793	WB-21850	37	NaN	NaN	NaN
Customer Name	9994	793	William Brown	37	NaN	NaN	NaN
Segment	9994	3	Consumer	5191	NaN	NaN	NaN
Country	9994	1	United States	9994	NaN	NaN	NaN
City	9994	531	New York City	915	NaN	NaN	NaN
State	9994	49	California	2001	NaN	NaN	NaN
Postal Code	9994.0	NaN	NaN	NaN	55190.379428	32063.69335	1040.0
Region	9994	4	West	3203	NaN	NaN	NaN
Product ID	9994	1862	OFF-PA-10001970	19	NaN	NaN	NaN
Category	9994	3	Office Supplies	6026	NaN	NaN	NaN
Sub-Category	9994	17	Binders	1523	NaN	NaN	NaN
Product Name	9994	1850	Staple envelope	48	NaN	NaN	NaN
Sales	9994.0	NaN	NaN	NaN	229.858001	623.245101	0.444
Quantity	9994.0	NaN	NaN	NaN	3.789574	2.22511	1.0
Discount	9994.0	NaN	NaN	NaN	0.156203	0.206452	0.0
Profit	9994.0	NaN	NaN	NaN	28.656896	234.260108	-6599.978

In [75]:

```

segment_sales = df.groupby('Segment')['Sales'].sum().sort_values(ascending=False)
segment_sales.plot(kind='bar', color='teal', title='Sales by Customer Segment')
plt.xlabel('Segment')
plt.ylabel('Total Sales')
plt.show()

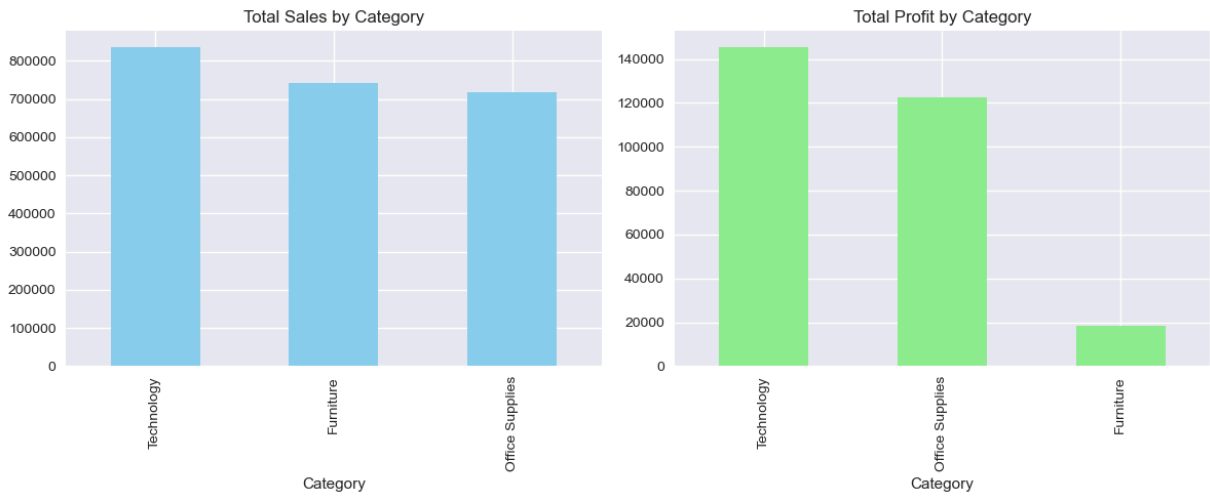
```



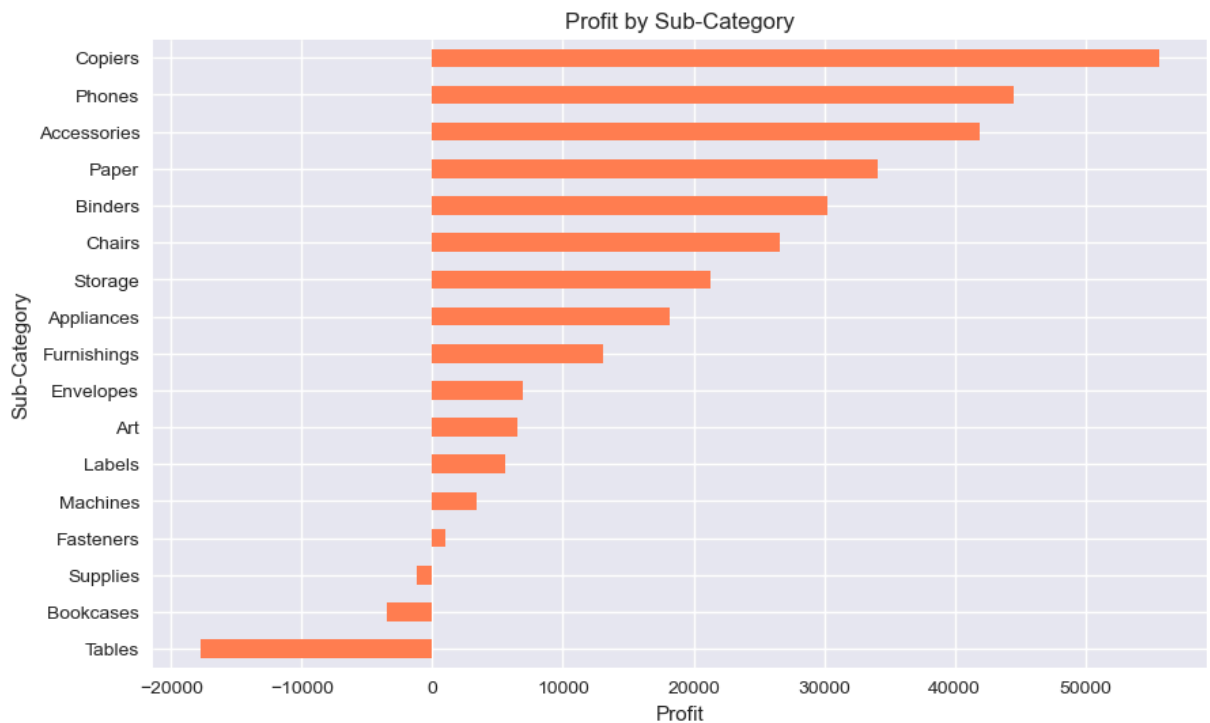
Customer Segment Analysis: Consumer and Corporate segments dominate total sales, while Home Office accounts for the smallest share.

```
In [64]: cat_sales = df.groupby('Category')['Sales'].sum().sort_values(ascending=False)
cat_profit = df.groupby('Category')['Profit'].sum().sort_values(ascending=False)

fig, ax = plt.subplots(1,2, figsize=(12,5))
cat_sales.plot(kind='bar', ax=ax[0], color='skyblue', title='Total Sales by')
cat_profit.plot(kind='bar', ax=ax[1], color='lightgreen', title='Total Profit by')
plt.tight_layout()
plt.show()
```



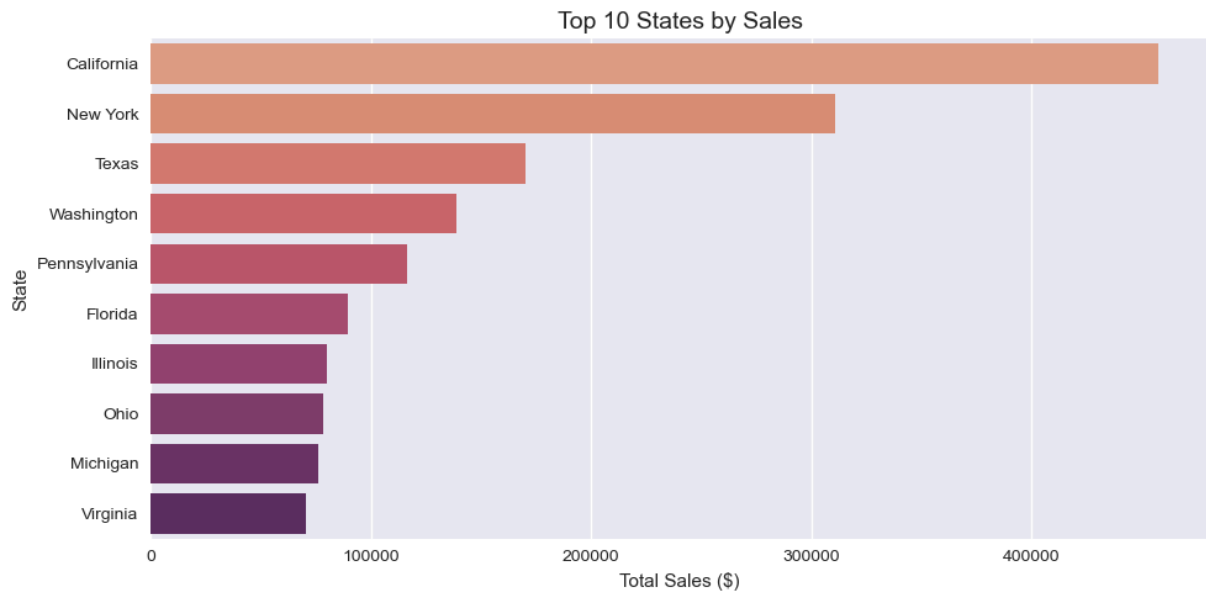
```
In [65]: subcat_profit = df.groupby('Sub-Category')['Profit'].sum().sort_values()
plt.figure(figsize=(10,6))
subcat_profit.plot(kind='barh', color='coral')
plt.title("Profit by Sub-Category")
plt.xlabel("Profit")
plt.ylabel("Sub-Category")
plt.show()
```



```
In [79]: state_sales = df.groupby('State')['Sales'].sum().sort_values(ascending=False)

plt.figure(figsize=(10,5))
sns.barplot(data=state_sales, x='Sales', y='State', hue='State', palette='fl
plt.title("Top 10 States by Sales", fontsize=14)
plt.xlabel("Total Sales ($)")
plt.ylabel("State")
plt.tight_layout()
plt.show()
```

state_sales



Out [79]:

	State	Sales
0	California	457687.6315
1	New York	310876.2710
2	Texas	170188.0458
3	Washington	138641.2700
4	Pennsylvania	116511.9140
5	Florida	89473.7080
6	Illinois	80166.1010
7	Ohio	78258.1360
8	Michigan	76269.6140
9	Virginia	70636.7200

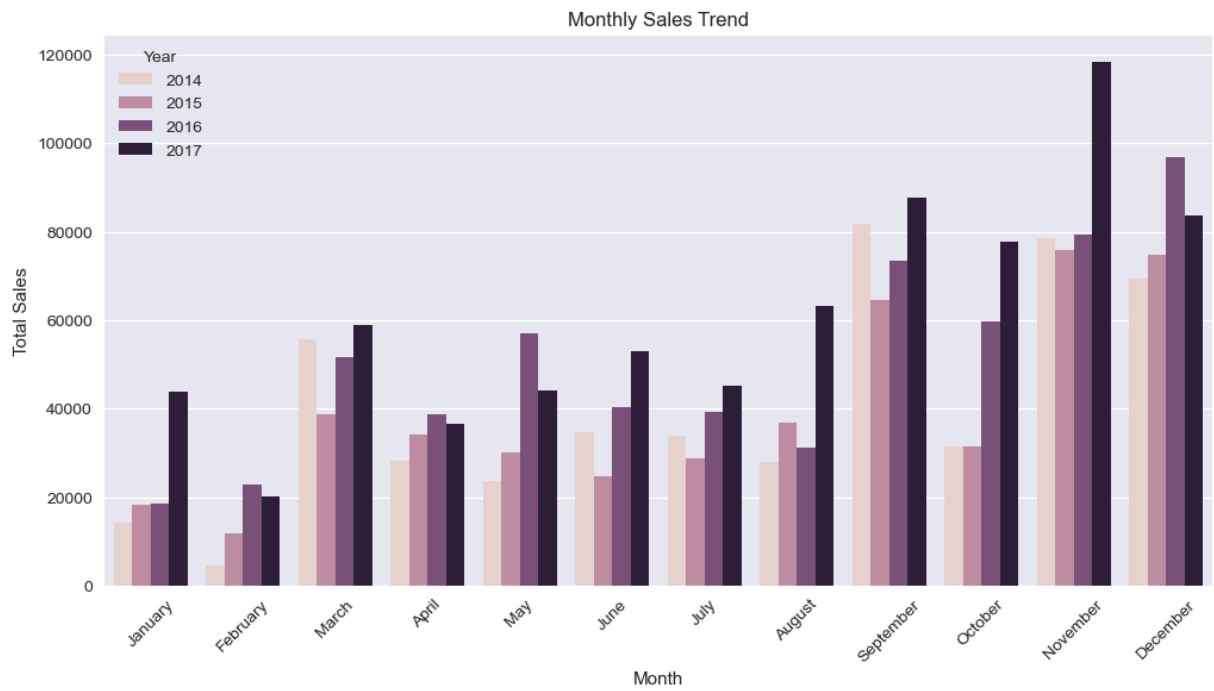
```
In [71]: import matplotlib.pyplot as plt
import seaborn as sns

month_order = ['January', 'February', 'March', 'April', 'May', 'June',
               'July', 'August', 'September', 'October', 'November', 'December']

plt.figure(figsize = (12,6))
sns.barplot(data = monthly_sales, x = 'Month', y = 'Sales', hue = 'Year', order = month_order)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Total Sales")
plt.xticks(rotation = 45)
plt.show()
```

<Figure size 1200x600 with 0 Axes>

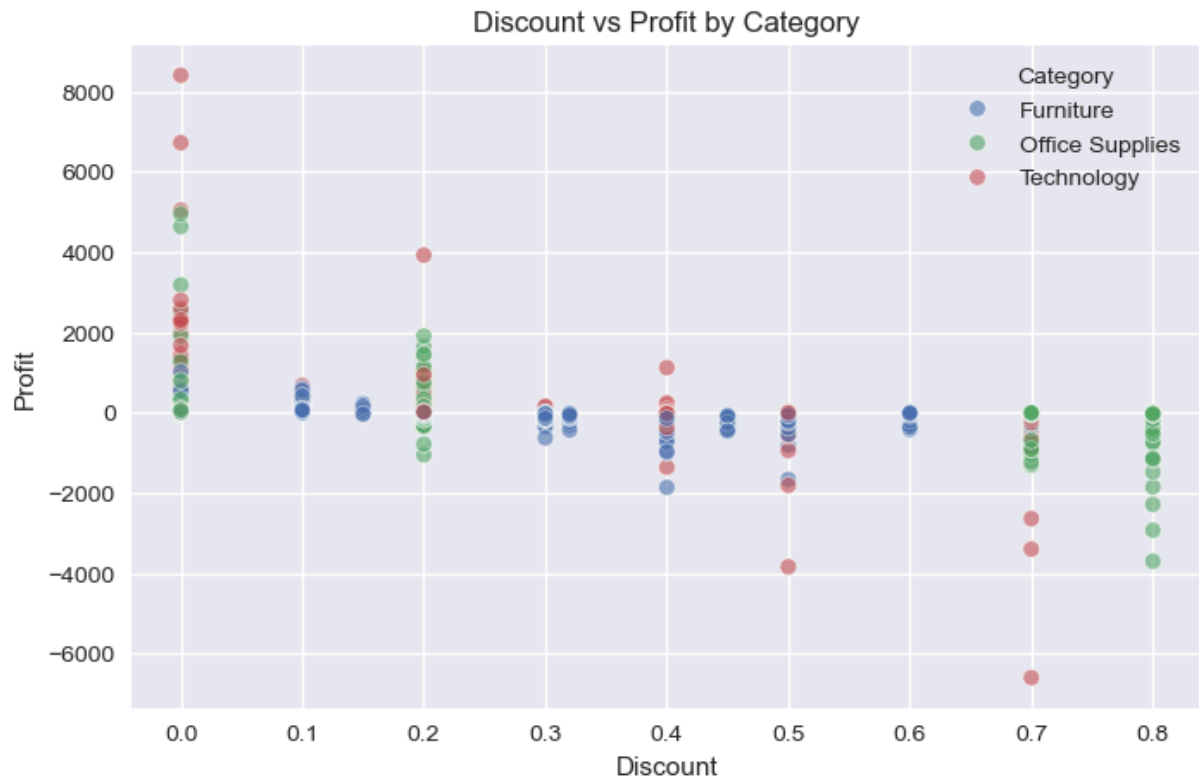
<Figure size 1200x600 with 0 Axes>

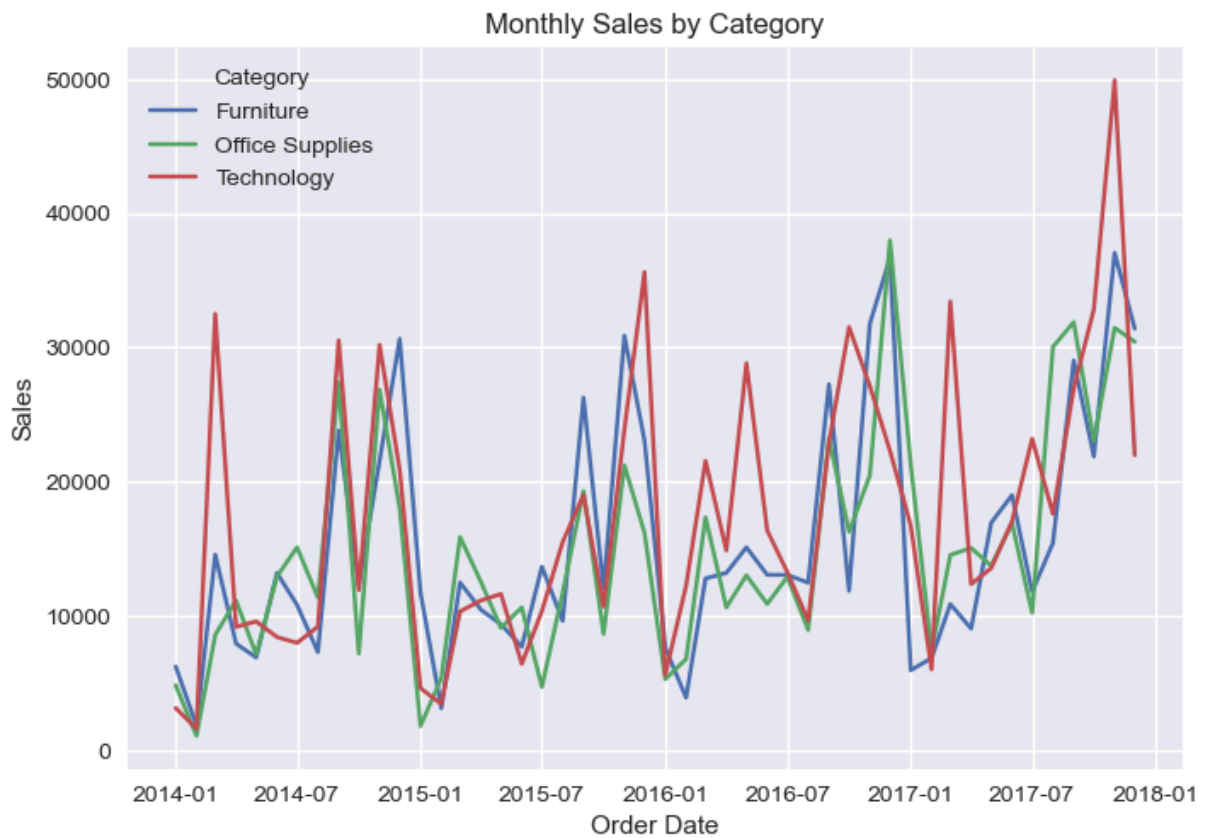


```
In [72]: region_sales = df.groupby('Region')['Sales'].sum().sort_values(ascending=False)
plt.figure(figsize=(6,4))
region_sales.plot(kind='bar', color='teal')
plt.title("Sales by Region")
plt.ylabel("Total Sales ($)")
plt.show()
```



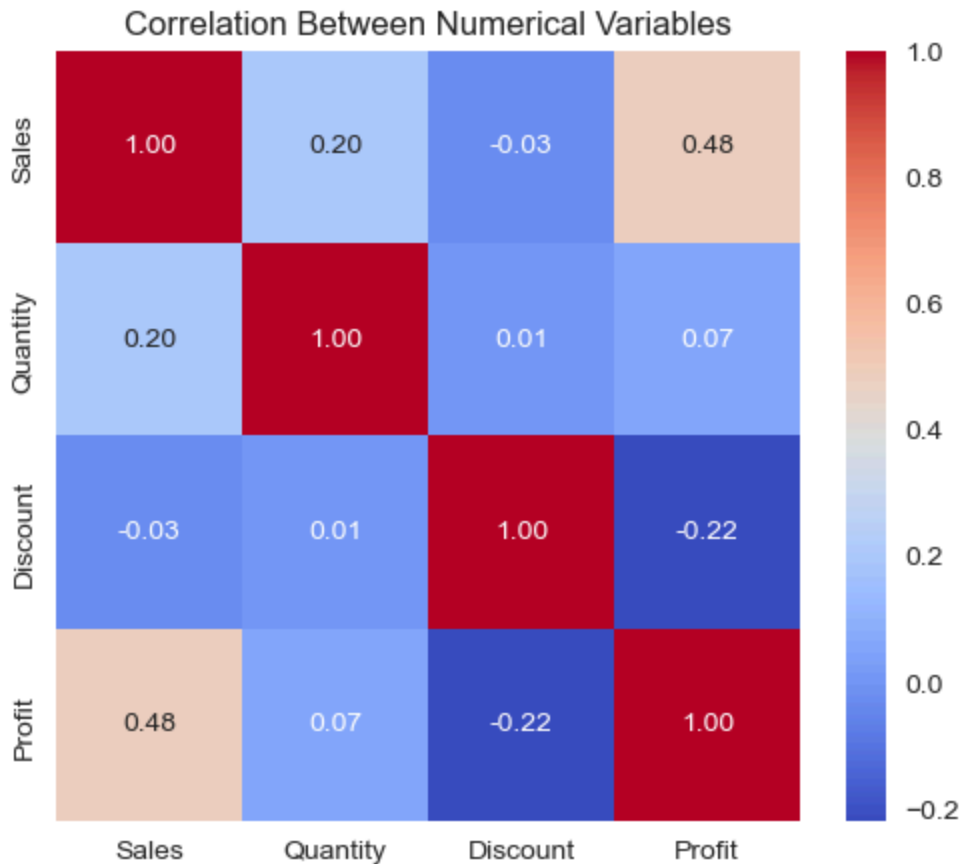
```
In [73]: plt.figure(figsize=(8,5))
sns.scatterplot(data=df, x='Discount', y='Profit', hue='Category', alpha=0.6)
plt.title("Discount vs Profit by Category")
plt.show()
```





Sales trend by Category over time: Technology is leading month on month

```
In [74]: plt.figure(figsize=(6,5))
sns.heatmap(df[['Sales','Quantity','Discount','Profit']].corr(), annot=True,
plt.title("Correlation Between Numerical Variables")
plt.show()
```



Insights

1. Top Performers: Technology and Office Supplies generated the highest revenue.
2. Regional Trends: The West region and California lead in total sales.
3. Seasonality: Sales peak around November–December, likely due to holiday promotions.
4. Customer Segments: Consumers and Corporate clients are the most profitable.
5. Discount Strategy: Moderate discounts improve sales; excessive ones reduce profit.

Recommendations

This analysis provides a clear overview of the Sample Superstore's sales performance. Leveraging these insights can help optimize discount strategies, improve regional targeting, and enhance overall profitability.

References

- Vivek468, "Sample - Superstore Dataset (Final)", Kaggle, 2019.
<https://www.kaggle.com/datasets/vivek468/superstore-dataset-final>
- Python libraries: pandas, matplotlib, seaborn (documentation).